

POST Request in FastAPI

This topic is one of the most important concepts in FastAPI because:

- GET requests usually **retrieve data**
- POST requests usually **send data to the server**

And the data sent to the server is called the **Request Body**.

This is exactly how:

- User registration works
- Login systems work
- ML prediction APIs work
- Database records are created

1. Recap

Before understanding POST requests, let's quickly review what you've learned.

GET Request

Used to retrieve data.

Example:

```
@app.get("/students")
def get_students():
    return {"students": ["Ali", "Ahmed"]}
```

Request:

```
/students
Response:
{
  "students": ["Ali", "Ahmed"]
}
```

Path Parameters

Used to identify a specific resource.

Example:

```
/students/5
Student ID = 5
```

Query Parameters

Used for filtering or optional values.

Example:

```
/students?age=20
```

Filter students by age.

Limitation of Path & Query Parameters

Suppose we want to create a new student.

Student information:

```
{
  "name": "Ali",
  "age": 22,
  "department": "Computer Science"
}
```

Can we reasonably send all of this through path parameters?

```
/students/Ali/22/ComputerScience
```

Not practical.

Can we send all of it using query parameters?

```
/students?name=Ali&age=22&department=ComputerScience
```

Possible, but not ideal.

For large or structured data, we need something better.

2. Plan of Action

The lesson introduces a new concept:

Request Body

Instead of sending data through the URL,
we send it inside the request itself.

Mental Model

Think of sending a package.

URL

Contains:

Where should it go?

Request Body

Contains:

What is inside the package?

3. Request Body

What is a Request Body?

A Request Body is the data sent by the client to the server.

Most modern APIs send request bodies in:

JSON format

Example:

```
{
  "name": "Ali",
  "age": 22,
  "department": "CS"
}
```

```
}
```

Definition

Request Body = Data sent inside a request.

Real-Life Example

Imagine a student registration form.

User enters:

- Name
- Age
- Department

When the user clicks Submit:

The browser sends:

```
{  
  "name": "Ali",  
  "age": 22,  
  "department": "CS"  
}
```

This JSON becomes the request body.

Why POST Requests Use Request Bodies

POST is used when:

- Creating data
- Sending large data
- Sending structured data

Examples:

Application	Data Sent
Registration	User details
Login	Username & Password
E-commerce	Order details
ML API	Prediction inputs
Chatbot	User message

FastAPI + Request Body

FastAPI uses Pydantic models to define request bodies.

Step 1: Create a Pydantic Model

```
from pydantic import BaseModel  
  
class Student(BaseModel):  
    name: str  
    age: int  
    department: str
```

What does this model do?

It defines the expected structure:

```
{
  "name": "string",
  "age": "integer",
  "department": "string"
}
```

Step 2: Use it in POST Request

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Student(BaseModel):
    name: str
    age: int
    department: str

@app.post("/students")
def create_student(student: Student):
    return student
```

What Happens Here?

FastAPI automatically:

1. Receives JSON

```
{
  "name": "Ali",
  "age": 22,
  "department": "CS"
}
```

2. Validates Data

Checks:

- name is string
- age is integer
- department is string

3. Creates Python Object

```
student.name
student.age
student.department
```

4. Returns Response

```
{
  "name": "Ali",
  "age": 22,
  "department": "CS"
}
```

Why Pydantic is Powerful Here

Suppose user sends:

```
{
  "name": "Ali",
  "age": "twenty-two",
  "department": "CS"
}
```

FastAPI immediately returns validation error.
No extra code needed.

Example Error Response

```
{
  "detail": [
    {
      "msg": "Input should be a valid integer"
    }
  ]
}
```

4. Code Demo

A typical demo follows these steps.

Full Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Student(BaseModel):
    name: str
    age: int
    department: str

@app.post("/students")
def create_student(student: Student):
```

```
return {  
  "message": "Student created",  
  "student": student  
}
```

Testing Through Swagger UI

Open:

```
http://127.0.0.1:8000/docs
```

FastAPI automatically creates:

- Input form
- Validation
- Documentation

Example Input

```
{  
  "name": "Ali",  
  "age": 22,  
  "department": "CS"  
}
```

Example Output

```
{  
  "message": "Student created",  
  "student": {  
    "name": "Ali",  
    "age": 22,  
    "department": "CS"  
  }  
}
```

Understanding Request Lifecycle

Let's follow the complete journey.

Step 1

Client sends:

POST /students

Step 2

Request Body:

```
{  
  "name": "Ali",  
  "age": 22,
```

```
"department": "CS"
}
```

Step 3

FastAPI receives request.

Step 4

Pydantic validates data.

Step 5

Python object is created.

Step 6

Function executes.

Step 7

Response returned as JSON.

GET vs POST

Feature	GET	POST
Purpose	Retrieve data	Send/Create data
Request Body	Usually No	Usually Yes
Data Visible in URL	Yes	No
Large Data Support	Poor	Excellent
Common Use	Fetch records	Create records

Path Params vs Query Params vs Request Body

Method	Example	Purpose
Path Parameter	/students/5	Identify resource
Query Parameter	/students?age=20	Filter resource
Request Body	JSON data	Send structured data

Real Machine Learning Example

Suppose you deploy a sentiment analysis model.

User sends:

```
{
  "text": "This movie is amazing!"
}
```

Request Body:

```
class Review(BaseModel):
```

```
    text: str
```

API:

```
@app.post("/predict")
```

```
def predict(review: Review):
    prediction = model(review.text)
    return {"prediction": prediction}
```

This is exactly how most ML APIs work.

Think:

Path Parameter

Which item?

/student/5

Query Parameter

How should I filter it?

/students?age=20

Request Body

What data am I sending?

```
{
  "name": "Ali",
  "age": 22
}
```

Final Takeaway

A **POST request** is used when you want to send data to the server.

That data is usually sent through a **Request Body**, typically in JSON format.

FastAPI works with Pydantic to:

- Receive the JSON
- Validate the data
- Convert it into Python objects
- Return clean responses

This Request Body + Pydantic combination is the foundation of nearly every modern FastAPI application, including Machine Learning APIs, user registration systems, CRUD applications, and production web services.